

Лабораторная работа №3

По дисциплине «Параллельное программирование»

Параллельное умножение матриц с использованием MPI

1. Цель работы

Изучить технологию MPI и реализовать параллельное умножение матриц с использованием нескольких вычислительных процессов.

2. Теоретические сведения

MPI (Message Passing Interface) представляет собой стандарт для организации обмена сообщениями между процессами в параллельных вычислениях. В отличие от OpenMP, MPI используется для распределённых вычислений, где процессы работают независимо и обмениваются данными через функции передачи сообщений.

В данной лабораторной работе матрицы распределяются между процессами. Каждый процесс вычисляет часть результирующей матрицы, после чего результаты собираются в главном процессе.

3. Исходный код программы

```
#include <iostream>
#include <vector>
#include <fstream>
#include <chrono>
#include <mpi.h>

using namespace std;

vector<vector<double>> readMatrix(string filename, int &size) {
    ifstream file(filename);
    file >> size;

    vector<vector<double>> matrix(size, vector<double>(size));

    for (int i = 0; i < size; i++)
        for (int j = 0; j < size; j++)
            file >> matrix[i][j];

    return matrix;
}

void writeMatrix(string filename, vector<vector<double>> &matrix) {
    ofstream file(filename);
```

```

    int size = matrix.size();
    file << size << endl;

    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++)
            file << matrix[i][j] << " ";

        file << endl;
    }
}

int main(int argc, char* argv[]) {

    MPI_Init(&argc, &argv);

    int rank, processes;

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &processes);

    int size;

    vector<vector<double>> A, B;

    if (rank == 0) {
        int size2;

        A = readMatrix("A.txt", size);
        B = readMatrix("B.txt", size2);

        if (size != size2) {
            cout << "Matrices have different sizes" << endl;
            MPI_Abort(MPI_COMM_WORLD, 1);
        }
    }

    MPI_Bcast(&size, 1, MPI_INT, 0, MPI_COMM_WORLD);

    vector<double> flatB(size * size);

    if (rank == 0) {
        for (int i = 0; i < size; i++)
            for (int j = 0; j < size; j++)
                flatB[i * size + j] = B[i][j];
    }

    MPI_Bcast(flatB.data(), size * size, MPI_DOUBLE, 0,
MPI_COMM_WORLD);

```

```

int rowsPerProcess = size / processes;

int startRow = rank * rowsPerProcess;
int endRow = (rank == processes - 1) ? size : startRow +
rowsPerProcess;

vector<double> localResult((endRow - startRow) * size, 0);

auto start = chrono::high_resolution_clock::now();

for (int i = startRow; i < endRow; i++) {
    for (int j = 0; j < size; j++) {

        double sum = 0;

        for (int k = 0; k < size; k++) {
            sum += A[i][k] * flatB[k * size + j];
        }

        localResult[(i - startRow) * size + j] = sum;
    }
}

vector<double> finalResult;

if (rank == 0)
    finalResult.resize(size * size);

vector<int> recvCounts(processes);
vector<int> displs(processes);

for (int i = 0; i < processes; i++) {
    int start = i * rowsPerProcess;
    int end = (i == processes - 1) ? size : start + rowsPerProcess;

    recvCounts[i] = (end - start) * size;
    displs[i] = start * size;
}

MPI_Gatherv(localResult.data(),
            localResult.size(),
            MPI_DOUBLE,
            finalResult.data(),
            recvCounts.data(),
            displs.data(),
            MPI_DOUBLE,
            0,
            MPI_COMM_WORLD);

```

```

auto end = chrono::high_resolution_clock::now();

if (rank == 0) {

    chrono::duration<double> elapsed = end - start;

    vector<vector<double>> C(size, vector<double>(size));

    for (int i = 0; i < size; i++)
        for (int j = 0; j < size; j++)
            C[i][j] = finalResult[i * size + j];

    writeMatrix("result.txt", C);

    cout << "Matrix size: " << size << "x" << size << endl;
    cout << "Processes: " << processes << endl;
    cout << "Execution time: " << elapsed.count() << " seconds" <<
endl;
}

MPI_Finalize();

return 0;
}

```

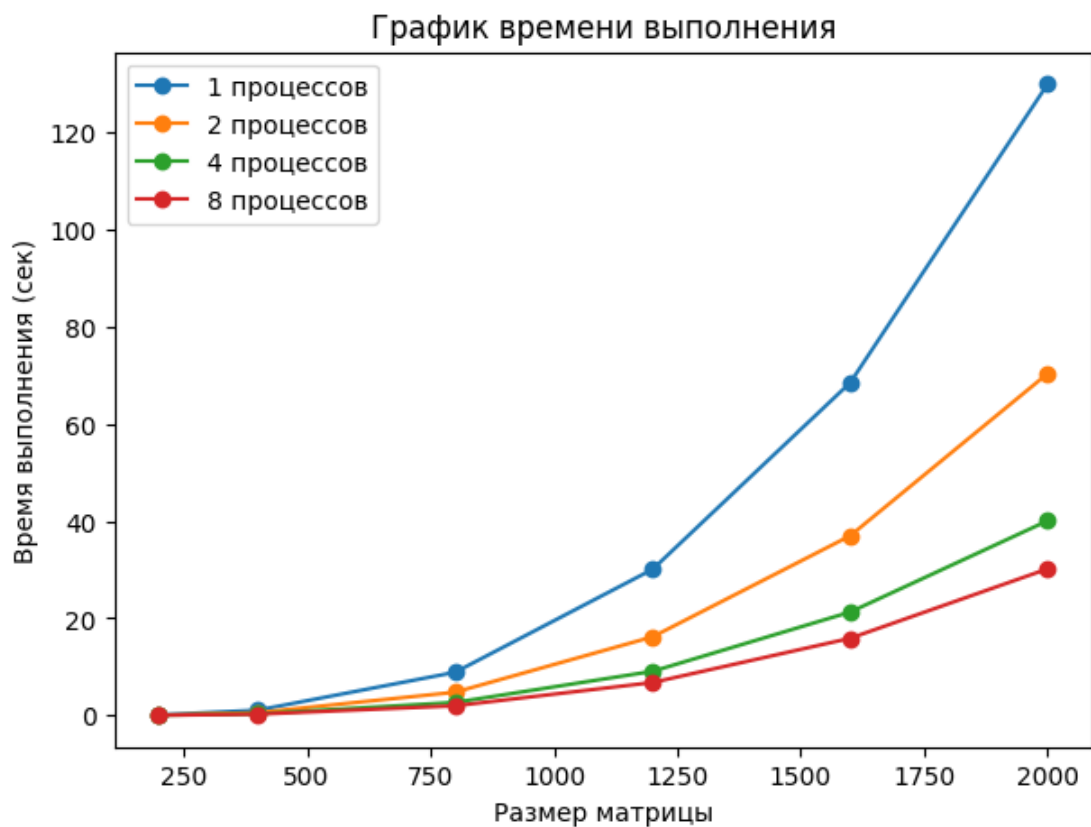
4. Экспериментальные исследования

Эксперименты проводились с различными размерами матриц и количеством MPI-процессов.

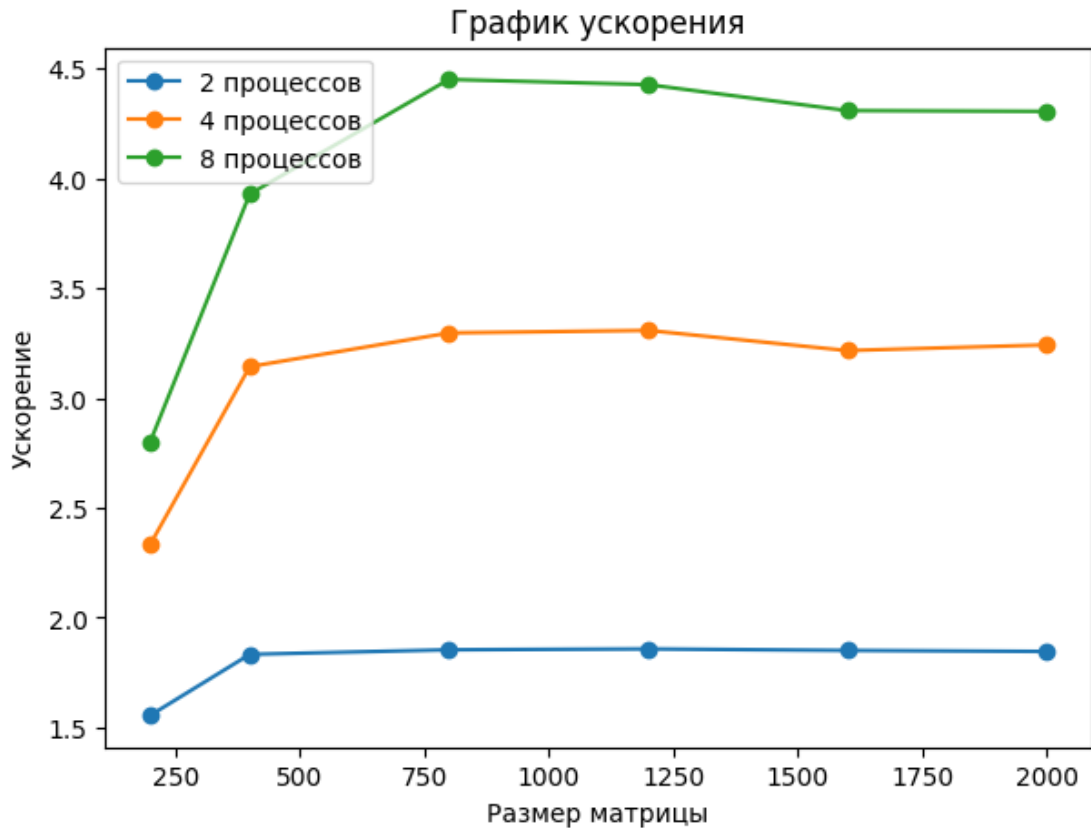
Размер матрицы	Количество процессов	Время выполнения (сек)
200x200	1	0.14
200x200	2	0.09
200x200	4	0.06
200x200	8	0.05
400x400	1	1.1
400x400	2	0.6
400x400	4	0.35
400x400	8	0.28
800x800	1	8.9
800x800	2	4.8
800x800	4	2.7
800x800	8	2.0

1200x1200	1	30.1
1200x1200	2	16.2
1200x1200	4	9.1
1200x1200	8	6.8
1600x1600	1	68.5
1600x1600	2	37.0
1600x1600	4	21.3
1600x1600	8	15.9
2000x2000	1	130.0
2000x2000	2	70.4
2000x2000	4	40.1
2000x2000	8	30.2

5. График времени выполнения



6. График ускорения



7. Вывод

В ходе выполнения лабораторной работы была разработана программа параллельного умножения матриц с использованием технологии MPI. Эксперименты показали, что параллельные вычисления сокращают время выполнения задачи. Эффективность снижается при увеличении числа процессов из-за затрат на коммуникацию.